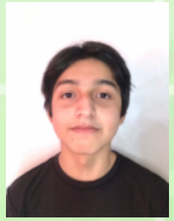
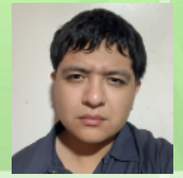
UNIVERSIDAD AUTONOMA DE NUEVO LEON
FACULTAD DE INGENIERIA MECANICA Y ELECTRICA

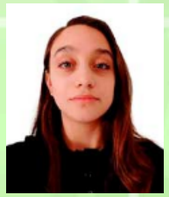
Almacenaje, memoria y archivos

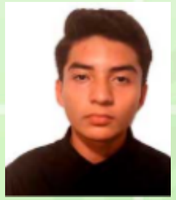
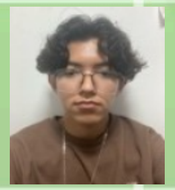
Sistemas Operativos

Dra: Norma Edith Marin Martinez
Equipo N°3

unidos
#VIVELAFIME
VIVE LA FIME

Nombre	Matricula	% de participacion
Camacho Moreno Jesus Emmanuel 	2153842	100 %
Cardenaz Vazquez Carlos Tomas	2226997	100 %
Cerda Herrera Juan Kristof 	2050864	100 %
Espindola Martinez Juan	2153695	100 %

Nombre	Matricula	% de participacion
Garza Arizpe Emilio	2143702	100 %
Garza Fernandez Vanessa Judith 	2145305	100 %
Gonzalez Martinez Mario Ivan	2117170	100 %
Gonzalez Parra Sylvia Yadira	2142962	100 %

Nombre	Matricula	% de participacion
Lopez Guzman Eder Abihu 	2116464	100 %
Peña Mendoza Cesar Maximiliano	2144135	100 %
Perez Ordaz Andres Daniel	2163200	100 %
Salazar Torres Isaias Leonardo 	2227016	100 %

Almacenaje, memoria y archivos

Entendemos como almacenamiento a cualquier medio físico capaz de retener datos, ya sea temporal o permanentemente. Se organiza en una jerarquía según velocidad, costo y volatilidad: desde los registros del CPU y la caché, pasando por la memoria principal, hasta el almacenamiento secundario (discos, SSD) y terciario (respaldos).

Bajo este contexto nos referimos a la memoria principal o RAM: el dispositivo de acceso directo y volátil donde residen las instrucciones y datos que la CPU está ejecutando activamente en un momento dado. Se distingue del almacenamiento secundario porque es mucho más rápida, pero pierde su contenido al apagar el equipo.

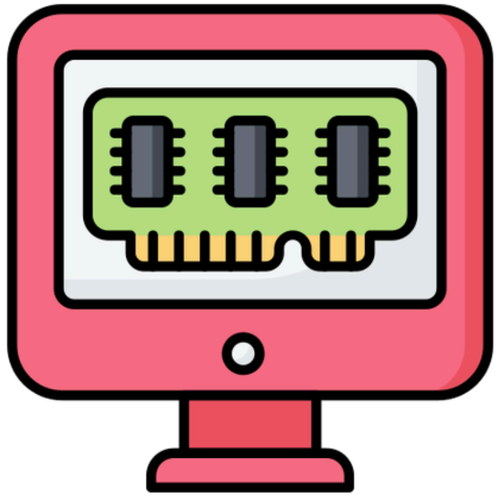
La unidad lógica de almacenamiento persistente que define el sistema operativo: una colección de datos con nombre que permanece guardada incluso después de que el proceso que la creó termina. El conjunto de archivos, junto con la estructura (directorios) que los organiza, forma el sistema de archivos.

Tipos de memorias

Memoria Física

En cualquier sistema informático, es fundamental distinguir claramente entre la memoria física y la memoria virtual. La memoria física, o RAM, es el recurso de hardware real y tangible donde se ejecutan los procesos, pero su capacidad es estrictamente limitada.

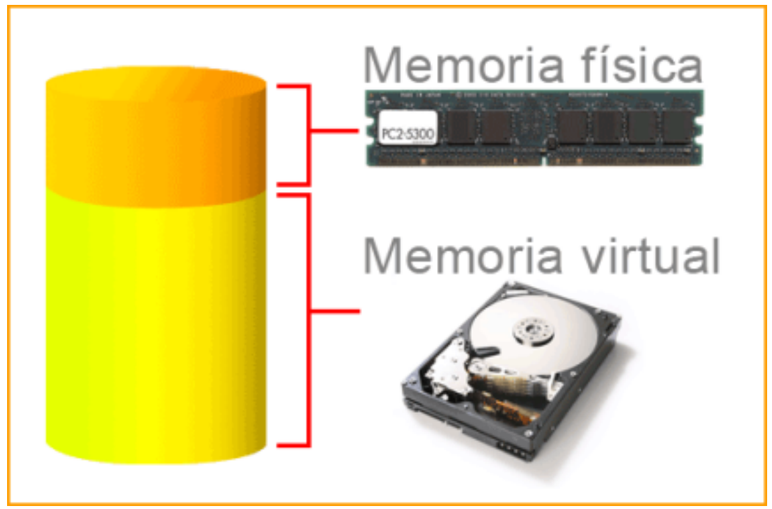
Memoria Virtual



Para romper esta limitación, los sistemas operativos modernos implementan la memoria virtual, una abstracción lógica que le hace creer a cada proceso que tiene a su disposición un espacio de direcciones inmenso e independiente.

Conexión entre ambos

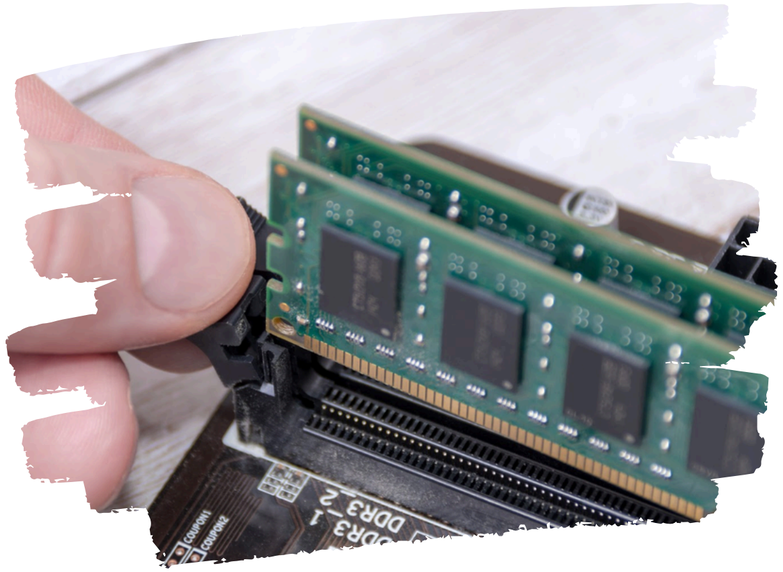
La conexión entre ambos mundos se logra mediante la paginación bajo demanda: el sistema mantiene en la memoria física solo las páginas exactas que se están utilizando en ese instante, dejando el resto de la información en el almacenamiento y trayéndola a la RAM únicamente cuando es necesaria.



Memoria en sistemas de escritorio

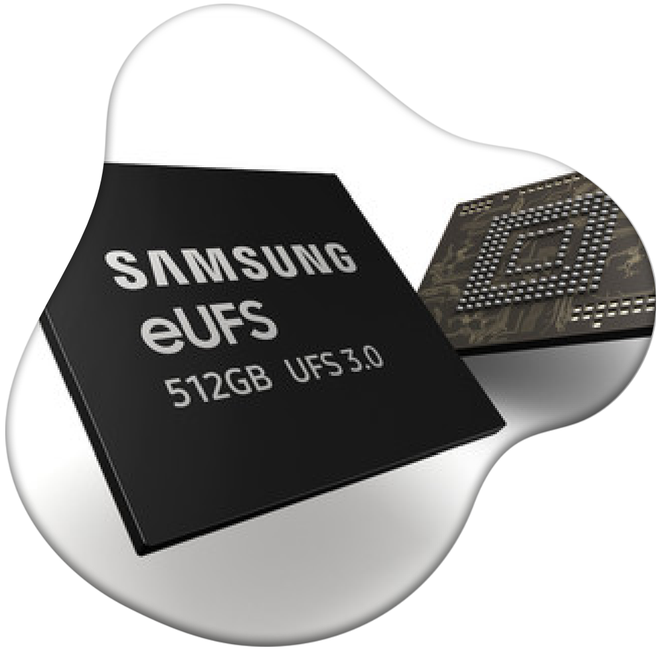
En los sistemas operativos de escritorio, la necesidad de liberar espacio se resuelve casi siempre utilizando una variante del algoritmo del reloj.

Esta estrategia es la favorita en estos entornos porque ofrece un excelente equilibrio técnico, logrando aproximarse al alto rendimiento del modelo ideal LRU (Least Recently Used) mientras se encarga de mantener una sobrecarga de cómputo bastante baja, evitando así saturar los recursos del procesador.



Memoria en sistemas móviles

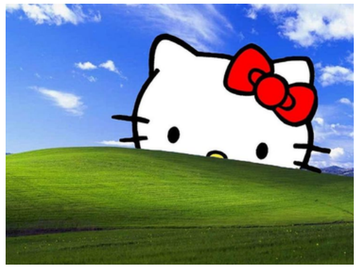
En los ecosistemas móviles el reto cambia de naturaleza debido a que la memoria flash se desgasta físicamente con cada ciclo de escritura. Para alargar la vida útil del hardware, ninguno de los sistemas móviles principales recurre a una partición de intercambio tradicional en disco.



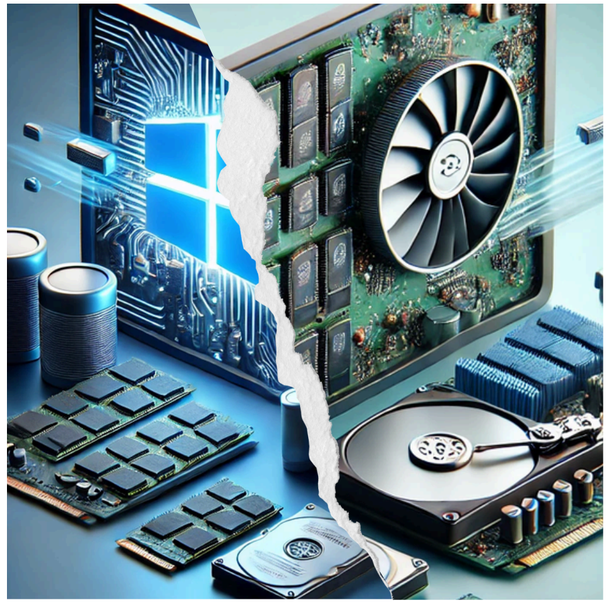
En su lugar, el problema se aborda primero comprimiendo las páginas inactivas dentro de la propia memoria RAM y, si esta medida no basta para liberar espacio, el sistema toma la decisión agresiva de terminar procesos completos.

Gestión de memoria y sistemas de archivos en sistemas operativos de escritorio

Gestión de memoria en Windows



En los sistemas Windows, la memoria se gestiona mediante paginación bajo demanda con clustering, lo que significa que, ante un fallo de página, el sistema no carga solo una, sino varias páginas sucesivas anticipando su uso futuro.



Cada proceso recibe un conjunto de trabajo con un límite mínimo y máximo de páginas, el cual se ajusta dinámicamente si la memoria escasea. Para liberar espacio de forma eficiente, el sistema emplea una variante del algoritmo del reloj en monoprocesadores, o el algoritmo FIFO en entornos multiprocesador.

Sistemas de archivos en Windows

Para el almacenamiento local, Windows utiliza NTFS, un sistema que trata absolutamente todo como un objeto estructurado con atributos.

La información de los archivos se organiza detalladamente en la Tabla Maestra de Archivos (MFT), utilizando números lógicos de clúster (LCN) para ubicar los datos físicos en el disco. Además, los directorios se indexan mediante árboles B+, una estructura que garantiza búsquedas muy rápidas sin requerir reorganizaciones constantes.

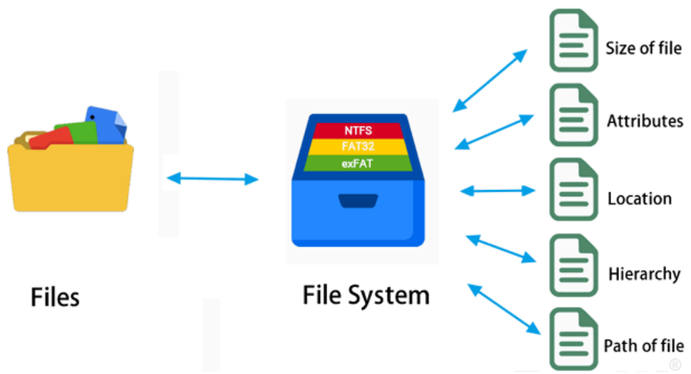
Gestión de memoria en Solaris

Solaris gestiona la memoria a través del proceso pageout, el cual opera mediante un algoritmo de reloj de dos manecillas: la manecilla frontal desactiva el bit de referencia de las páginas y la posterior evalúa si deben liberarse.

Este mecanismo se activa automáticamente cuando la memoria disponible cae por debajo del umbral lotsfree, adaptando la velocidad de exploración según la urgencia. Si el nivel crítico de memoria se mantiene durante varios segundos, el sistema adopta una medida más drástica y comienza a descargar procesos completos.

UNIX

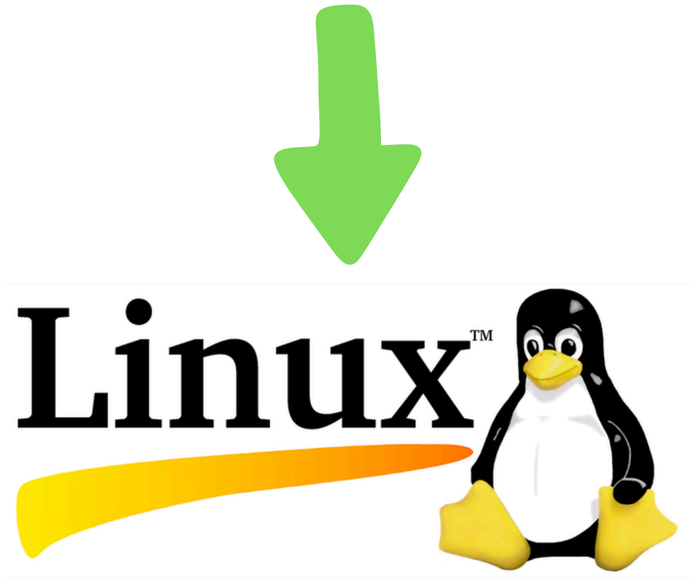
Sistemas de archivos en Solaris



Fiel a su herencia de UNIX, Solaris organiza los archivos utilizando nodos-i (inodes), una estructura de control que mantiene los metadatos completamente separados del nombre del archivo, lo que facilita la creación de múltiples enlaces hacia los mismos datos.

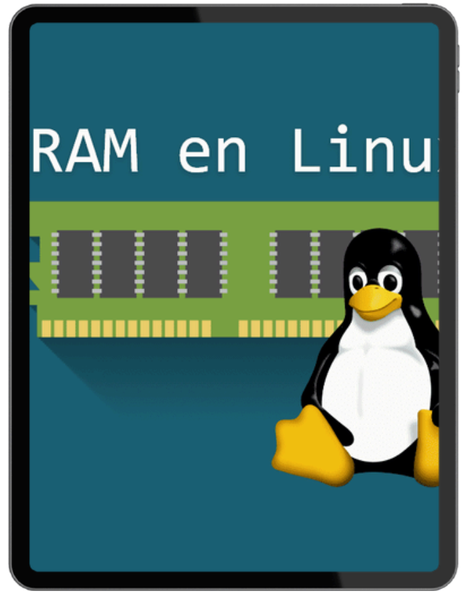
Para optimizar el rendimiento en discos magnéticos, arquitecturas clásicas como el Fast File System dividen las particiones en grupos de cilindros, reduciendo significativamente el desplazamiento físico de los cabezales de lectura.

Gestión de memoria en



En Linux, la memoria física se organiza en zonas específicas para adaptarse a las limitaciones de acceso directo a memoria (DMA) del hardware. Para administrar bloques de páginas contiguas, el núcleo utiliza el algoritmo de descomposición binaria denominado buddy system.

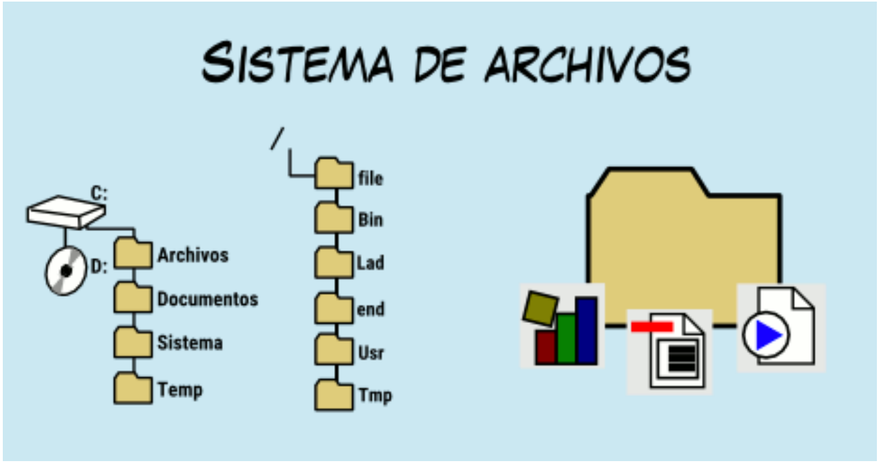
Adicionalmente, para gestionar la memoria dinámica de vida corta, el sistema emplea una asignación por franjas que recicla constantemente las estructuras y descriptores previamente liberados, optimizando la reutilización de recursos.



Sistemas de archivos en Linux

En cuanto al almacenamiento, Linux destaca por incorporar el Sistema de Archivos Virtual (VFS), una capa de abstracción orientada a objetos. Esta arquitectura le permite interactuar con diversos formatos de disco y red de manera transparente para el usuario.

Gracias a esto, el sistema puede almacenar en caché los nodos-i de forma unificada en la memoria, acelerando las lecturas futuras independientemente de si el archivo utiliza el formato nativo EXT2 o un sistema de archivos moderno estructurado en bitácora.

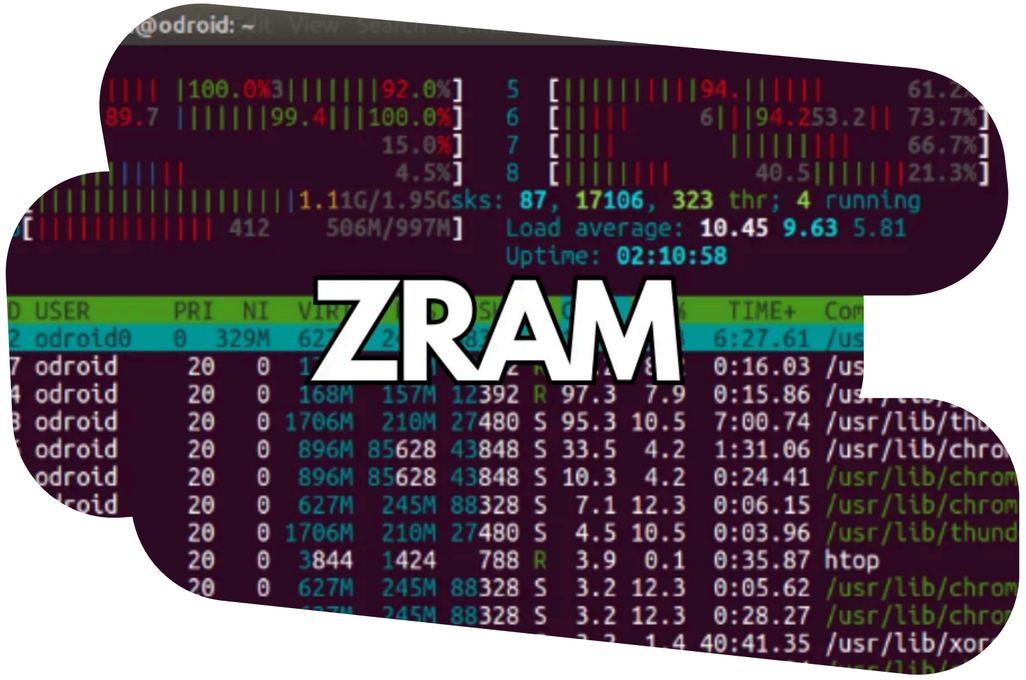


Gestión de memoria y sistemas de archivos en sistemas operativos móviles

Gestión de memoria en Android

En el ecosistema móvil, Android hereda la paginación de Linux, pero evita la partición de intercambio tradicional para no acortar la vida útil de la memoria flash. En su lugar, utiliza zRAM para comprimir las páginas inactivas directamente en la memoria RAM.

Además, su entorno de ejecución ART gestiona la memoria activa mediante un recolector de basura, y cuenta con mecanismos escalonados como el Low Memory Killer (LMK) para cerrar procesos en segundo plano cuando los recursos realmente escasean.



Sistemas de archivos en

Android

Para el almacenamiento, Android soporta múltiples formatos, siendo ext4 el estándar tradicional. Sin embargo, destaca la adopción progresiva de F2FS, un sistema de archivos diseñado desde cero para convivir con la naturaleza de las memorias flash.

Este sistema opera escribiendo los datos de forma secuencial al final de una cola, lo que reduce significativamente la sobrecarga de escritura y mejora el rendimiento general del dispositivo.

Ext4
File System

Gestión de memoria en iOS

A diferencia de otros sistemas móviles, iOS administra la memoria mediante el recuento automático de referencias (ARC), destruyendo y liberando objetos de manera inmediata en cuanto dejan de utilizarse, sin la necesidad de depender de un recolector de basura en segundo plano.

Cuando se presenta una escasez crítica de RAM, el sistema recurre a la compresión de páginas y activa el proceso Jetsam, un mecanismo del núcleo que cierra aplicaciones completas al instante para liberar espacio, evitando por completo el uso de particiones de intercambio en disco.

Sistemas de archivos en iOS

Para el almacenamiento local, el ecosistema de Apple utiliza APFS, un sistema de archivos diseñado específicamente para optimizar el rendimiento de las unidades de estado sólido.

Entre sus características principales destacan los clones, que permiten duplicar archivos sin ocupar espacio adicional hasta que sufren modificaciones reales, y las instantáneas, las cuales toman registros de solo lectura de todo el sistema para garantizar restauraciones seguras y automáticas durante las actualizaciones.

Gestión de memoria en HarmonyOS

En su versión NEXT basada en el microkernel HongMeng, HarmonyOS aborda la gestión de memoria aislando los espacios de trabajo de cada servicio dentro del espacio de usuario.

Para garantizar máxima fluidez, emplea un motor de latencia determinista que asigna y prioriza la memoria en tiempo real, evitando que los procesos compitan de forma desordenada por los recursos y asegurando que un fallo de memoria en un módulo no colapse al resto del sistema.

Sistemas de archivos en HarmonyOS

En el apartado de almacenamiento, el sistema de archivos de HarmonyOS incorpora cifrado nativo por hardware mediante la tecnología TrustZone.

Al ejecutarse de forma completamente aislada dentro del espacio de usuario, se asegura de que cualquier fallo o corrupción dentro de la capa de archivos quede contenido sin poner en riesgo la estabilidad del sistema operativo.

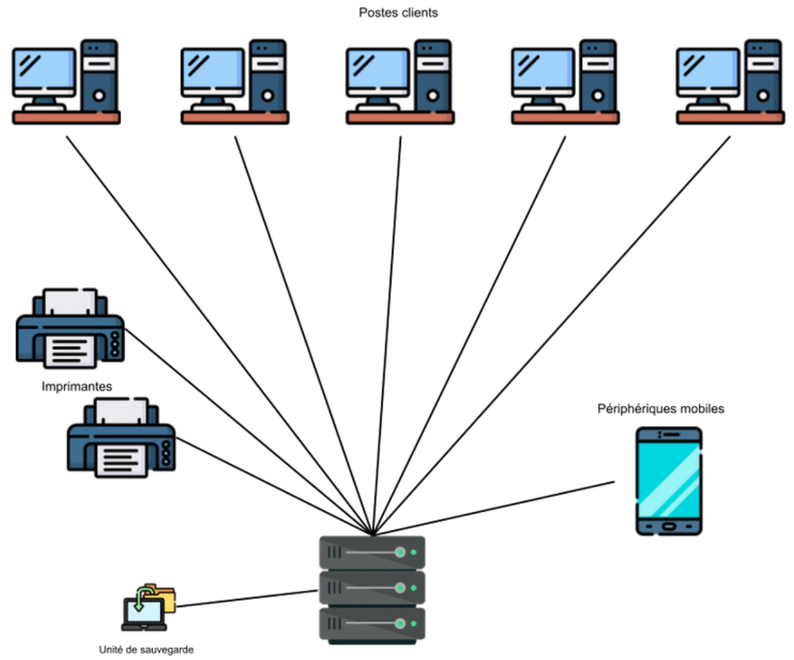
TrustZone
Your Trustable Learning Partner

HarmonyOS

Sistemas operativos de Red

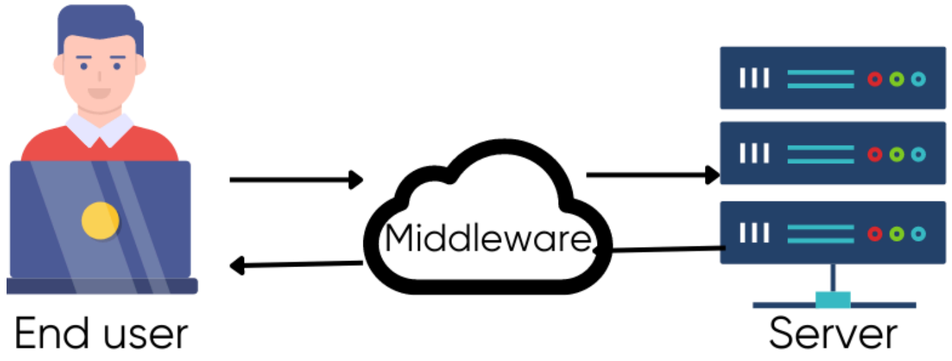
Los sistemas operativos de red operan bajo un paradigma diferente al no gestionar la memoria o los archivos físicos locales, sino funcionando como una capa de software superpuesta a los sistemas tradicionales para conectar computadoras independientes y compartir recursos.

Su característica fundamental es la transparencia limitada, donde cada nodo conserva su propia identidad y el usuario está consciente de que interactúa con múltiples máquinas. Este modelo contrasta con el de los sistemas operativos distribuidos, cuyo objetivo clásico era ocultar por completo la complejidad de la red para simular un único equipo centralizado.



Con el tiempo, la idea de un sistema distribuido puro perdió protagonismo en la industria frente al uso del "middleware", una solución mucho más pragmática y flexible.

Tecnologías como CORBA o DCOM de Microsoft demostraron que resultaba mucho más eficiente instalar una capa de integración sobre los sistemas operativos ya existentes para conectar los equipos de manera robusta, evitando la necesidad de reescribir desde cero toda la arquitectura del sistema.



Importancia de la administración de la memoria

Importancia de la administración de memoria

La administración de memoria es fundamental para que el sistema operativo mantenga un control ordenado y eficiente en la asignación de recursos. Su objetivo principal es responder a las exigencias de los entornos informáticos actuales, permitiendo un manejo flexible de los datos y dando soporte a la programación modular.

Para lograr este nivel de control y garantizar que el equipo funcione de manera óptima, el sistema operativo asume cinco responsabilidades fundamentales en la gestión del almacenamiento, las cuales desglosaremos a continuación.

Aislamiento de procesos

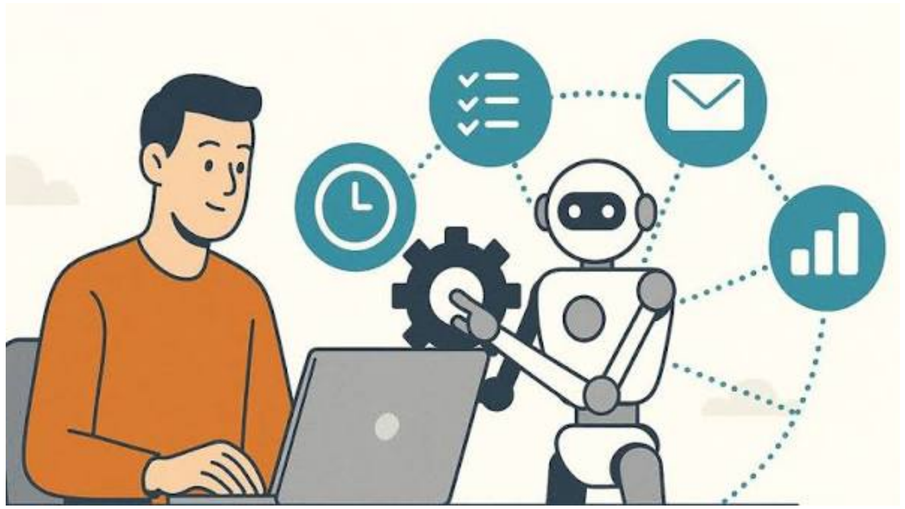
El sistema operativo debe garantizar que cada programa se ejecute en su propio espacio sin interferencias externas.

Esta función evita estrictamente que un proceso independiente invada o altere la memoria reservada para otro, resguardando tanto las instrucciones del programa como sus datos.

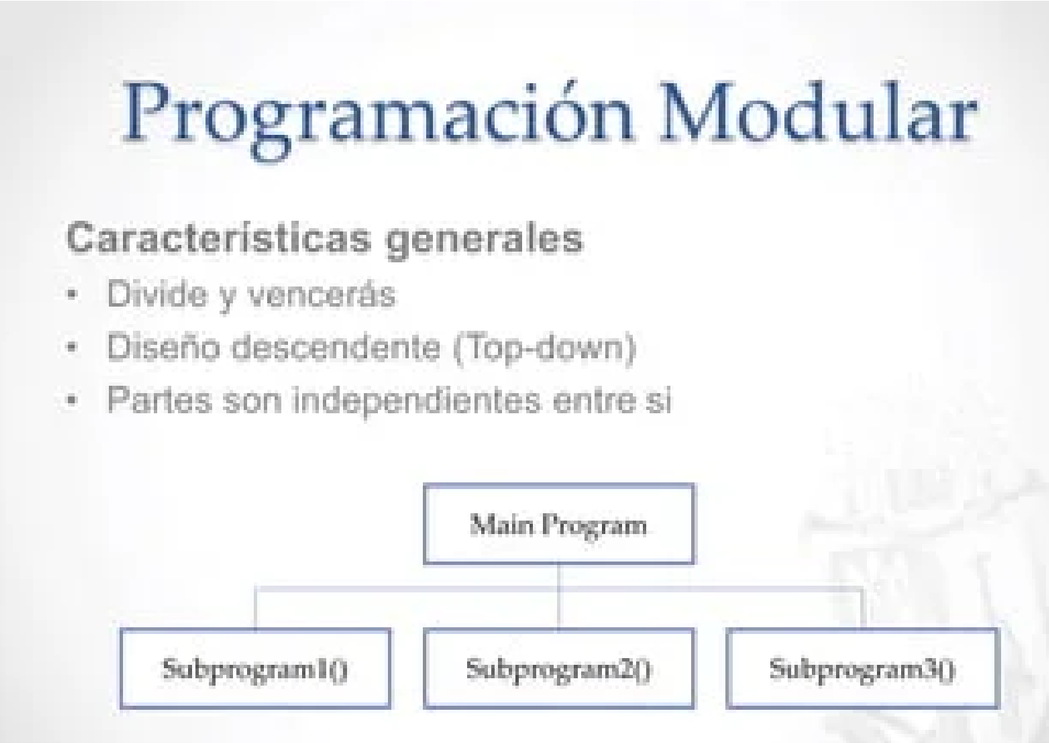
Asignación y gestión automática

Para elevar la eficiencia del equipo, la memoria se asigna a las tareas únicamente en el instante preciso en que la necesitan.

Todo este proceso ocurre de manera transparente para el programador, liberándolo de la carga de gestionar la memoria de forma manual.



Soporte a la programación modular



El sistema operativo debe ofrecer la flexibilidad necesaria para trabajar con estructuras de código independientes.

Esto permite a los desarrolladores definir módulos de programación con la libertad de crearlos, destruirlos o cambiar su tamaño de forma dinámica durante la ejecución.

Protección y control de accesos

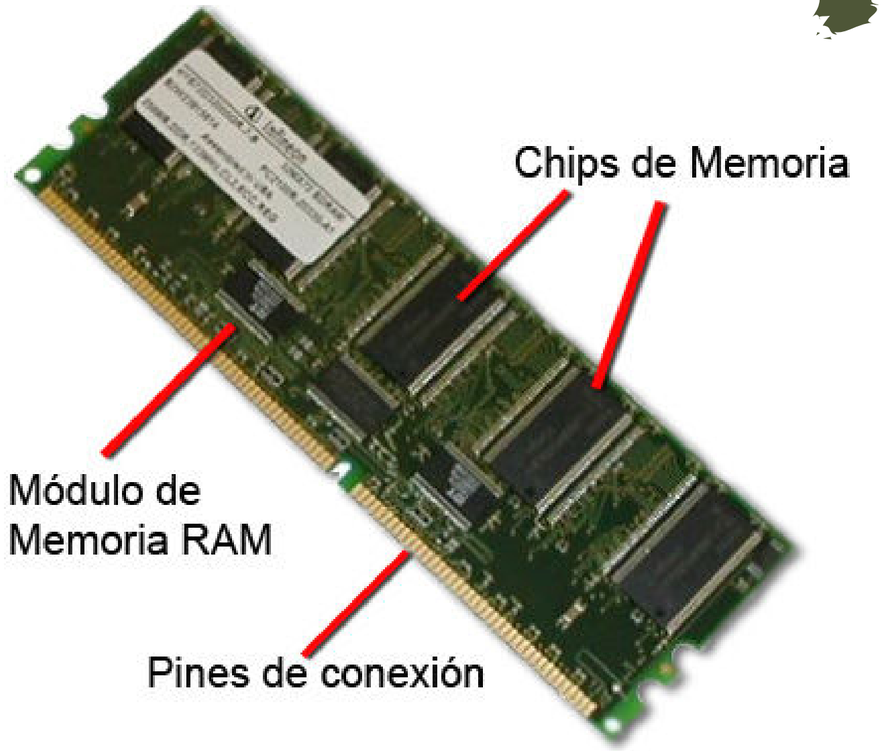
Esta función regula la forma en que múltiples usuarios o programas interactúan con la memoria.

El sistema operativo aplica controles de acceso para permitir distintas formas de uso, protegiendo al equipo contra amenazas que pongan en riesgo la integridad de las aplicaciones o del propio sistema.

Almacenamiento a largo plazo

A diferencia de la *memoria volátil*, esta responsabilidad asegura que la información se conserve de manera persistente, incluso después de apagar el equipo.

Esto se logra combinando el uso de la memoria virtual con estructuras permanentes como los archivos o ficheros.



Importancia de la memoria virtual

La memoria virtual es crucial porque permite que múltiples trabajos de usuario residan en la memoria principal de forma concurrente, eliminando los tiempos de espera entre la ejecución de procesos sucesivos.

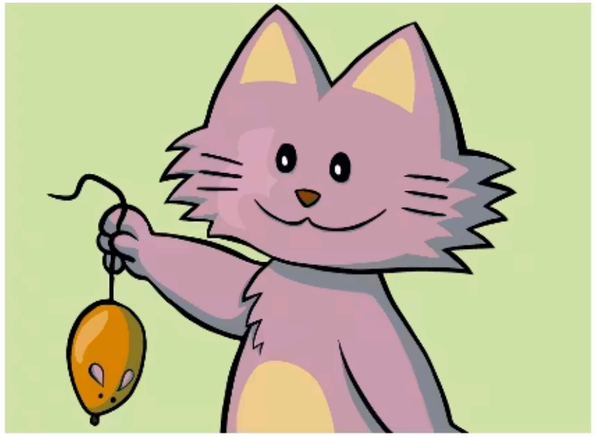
Gracias al sistema de paginación y al hardware de proyección dinámica, el sistema operativo traduce de manera transparente las direcciones virtuales a direcciones físicas reales en la RAM, logrando un mecanismo con muy baja sobrecarga computacional que optimiza al máximo el tráfico de datos en la jerarquía de memoria.

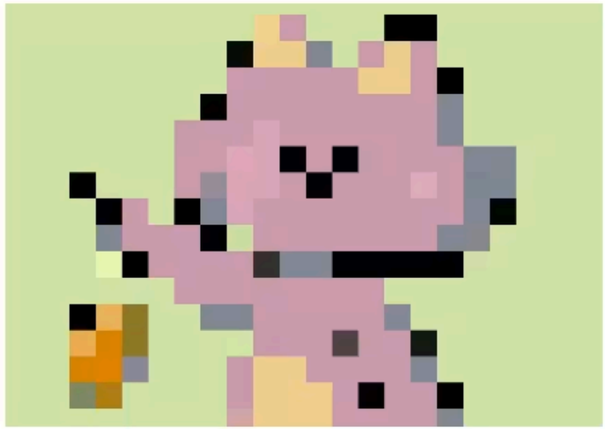
Control de los espacios disponibles y ocupados en memoria

Métodos de registro de memoria

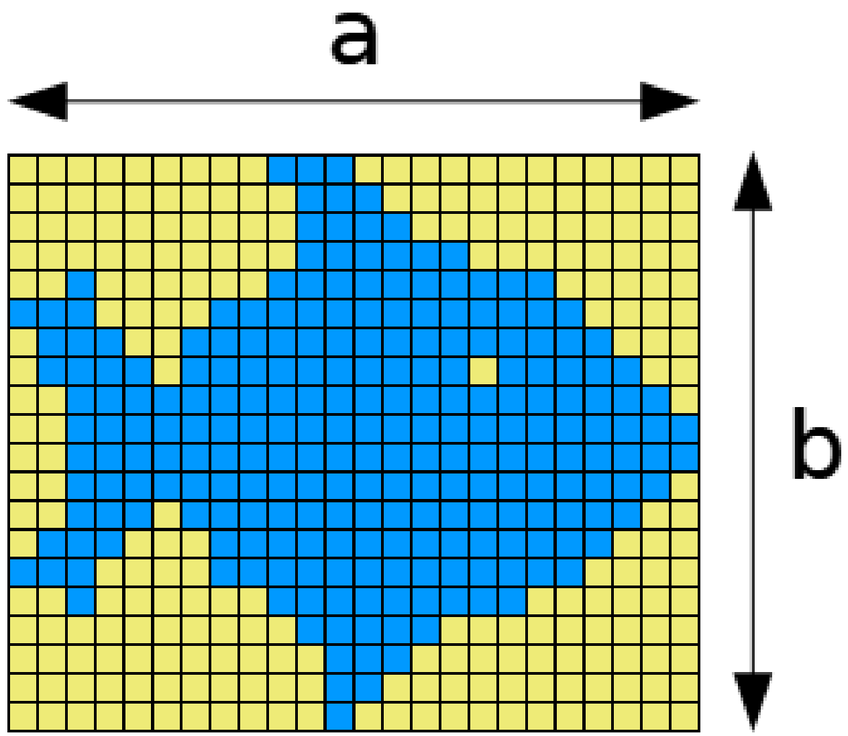
Para llevar un control ordenado de la memoria asignada y disponible, los sistemas operativos recurren principalmente a dos técnicas de administración: el mapa de bits y las listas libres o ligadas.

Ambas alternativas ofrecen una solución estructurada para que el administrador de memoria reconozca en todo momento qué bloques están en uso por algún proceso y cuáles permanecen vacíos.





Registro mediante mapa de bits



En un mapa de bits, la memoria se divide en unidades de asignación fijas y a cada una se le asigna un bit individual, donde un cero indica que el bloque está libre y un uno que está ocupado.

Este método ofrece una forma simple de registro cuyo tamaño total depende de la memoria y del tamaño de la unidad elegida; por ello, definir el tamaño de la unidad es vital, ya que unidades más pequeñas generan mapas de bits considerablemente más grandes.

Registro mediante listas ligadas

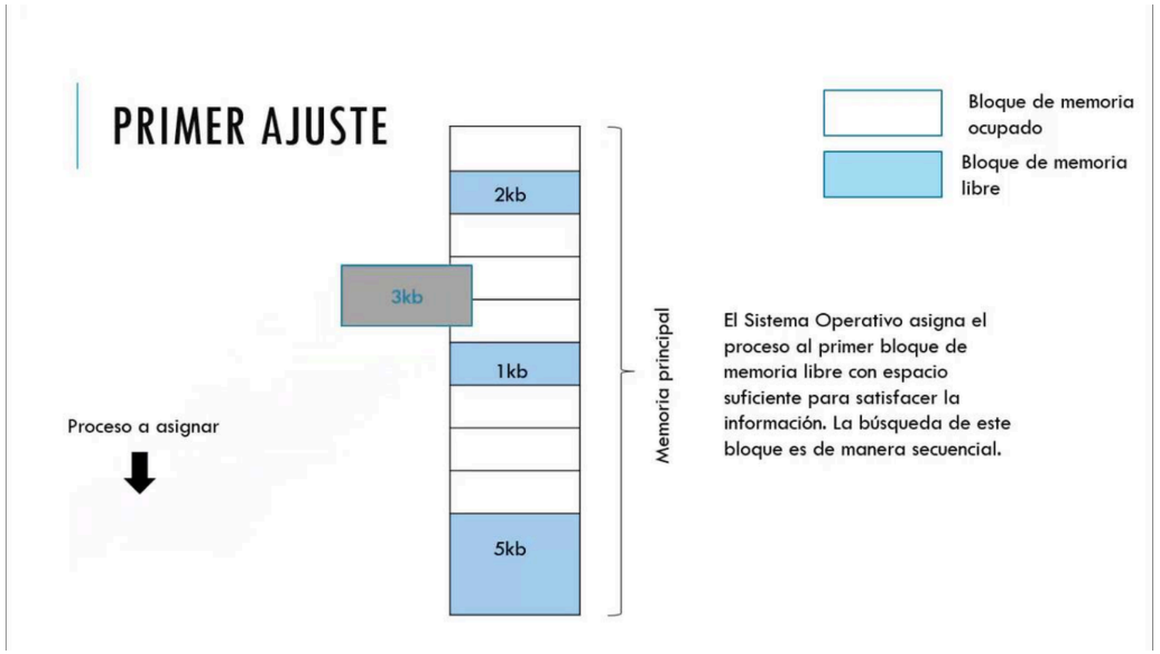
La otra alternativa consiste en mantener una lista ligada de segmentos que registra tanto los procesos activos como los huecos vacíos entre ellos.

Se recomienda mantener esta lista estrictamente ordenada por dirección de memoria, ya que esto simplifica la actualización del sistema al terminar o intercambiar un proceso y permite aplicar diversos algoritmos para la asignación de espacio.

Algoritmos de asignación: primer ajuste y siguiente ajuste

Para determinar dónde colocar un proceso, el primer ajuste recorre la lista y selecciona el primer hueco lo suficientemente grande, siendo un algoritmo rápido al buscar lo menos posible.

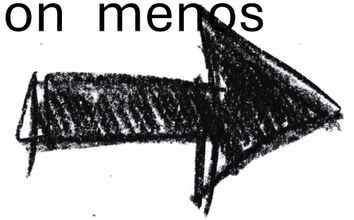
Por su parte, el siguiente ajuste funciona con la misma lógica, pero guarda un registro de su última ubicación para iniciar las búsquedas posteriores desde ese punto en lugar de empezar siempre desde el principio.

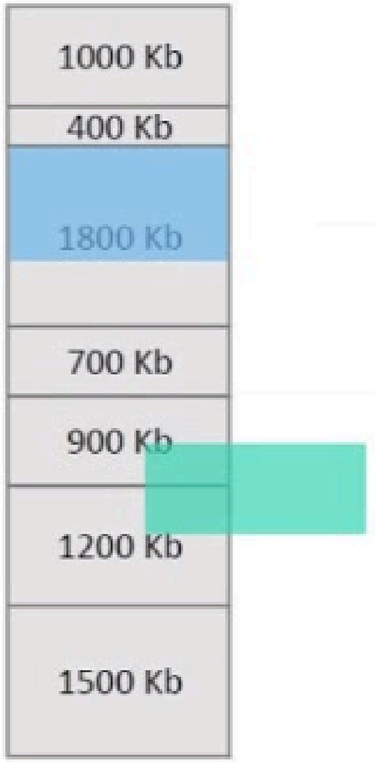


Algoritmos de asignación: mejor ajuste y peor ajuste

El algoritmo de mejor ajuste examina toda la lista para tomar el hueco más pequeño donde quepa el proceso, conservando los espacios más grandes para futuras necesidades a costa de ser más lento.

En contraposición, el peor ajuste selecciona siempre el hueco más grande disponible para intentar dejar un residuo que siga siendo útil, aunque en la práctica suele ser la opción menos óptima.

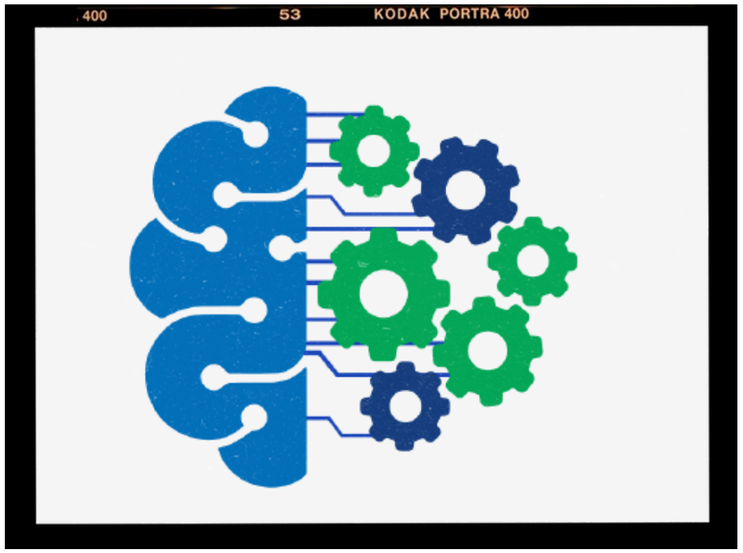




Optimización de la asignación

Para acelerar la ejecución de cualquiera de estos cuatro algoritmos, el administrador de memoria puede separar la estructura en dos listas independientes, una para los procesos y otra para los huecos.

Al enfocar toda la capacidad de búsqueda exclusivamente en la lista de espacios libres, se evita perder tiempo inspeccionando los segmentos de memoria que ya están ocupados.



Problemas frecuentes en la administración de memoria

Importancia de la administración

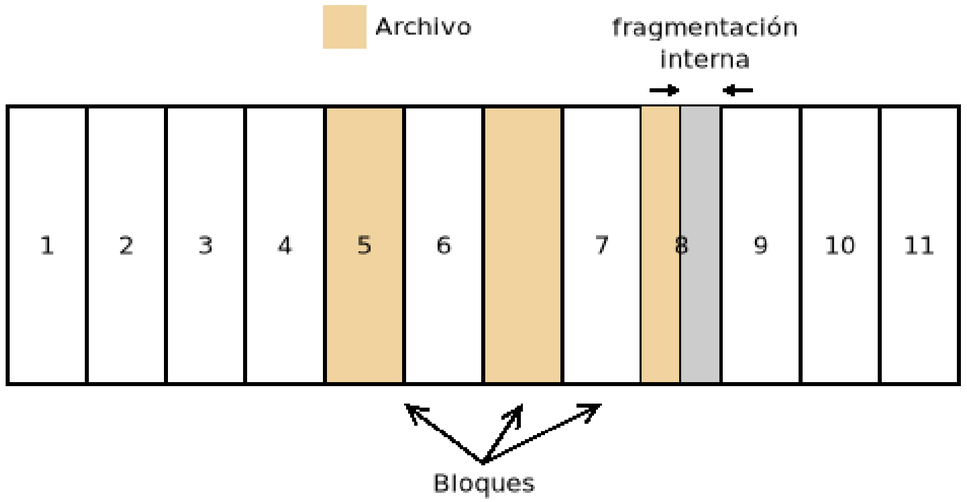
La administración de memoria es esencial para que el sistema operativo mantenga múltiples procesos funcionando simultáneamente.

Según Stallings, esta gestión debe cumplir con necesidades clave como la reubicación, la protección y el uso compartido de la información, llevando un control exacto de los espacios ocupados y disponibles, aunque intentar optimizar estos recursos siempre conlleva ciertos desafíos técnicos.

Problema de la fragmentación interna

Uno de los problemas más comunes en las particiones fijas es la fragmentación interna. Esto ocurre cuando un proceso requiere menos espacio del que se le asignó originalmente, dejando una parte del bloque sin utilizar de forma permanente.

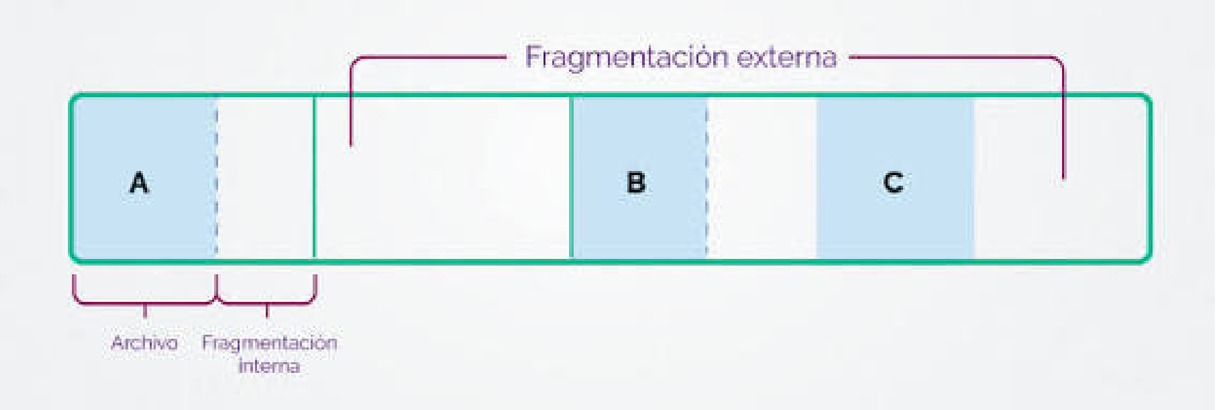
Aunque la pérdida parezca mínima individualmente, la suma de estos espacios desperdiciados a lo largo del sistema reduce significativamente la capacidad de la memoria.



Problema de la fragmentación externa

Por otro lado, el particionamiento dinámico suele generar fragmentación externa. A medida que los programas entran y salen de la memoria, se van creando pequeños huecos libres dispersos entre los datos ocupados.

El problema principal es que, aunque la suma de estos huecos sea suficiente para alojar una nueva tarea, al no existir un bloque continuo grande, el nuevo proceso no puede ejecutarse.



Estrategias de ubicación de procesos

Para decidir en qué espacio libre colocar cada tarea, el sistema utiliza algoritmos como First-fit, Next-fit o Best-fit.

Cada estrategia tiene sus propias ventajas y complicaciones; por ejemplo, el Best-fit intenta optimizar la memoria ocupando el hueco más pequeño donde quepa el proceso, pero a la larga termina dejando cientos de espacios diminutos que resultan totalmente inservibles.

Compactación como solución

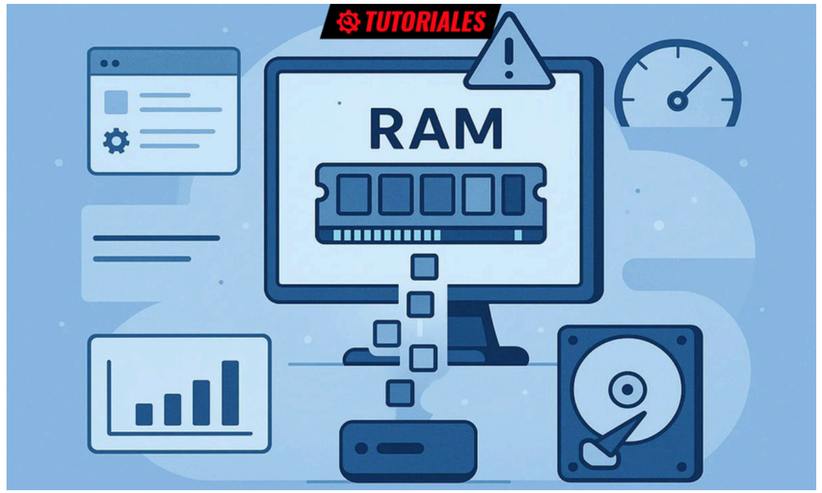
Una alternativa para resolver la fragmentación externa es aplicar la compactación. Esta técnica consiste en mover físicamente los procesos activos para agrupar todos los huecos dispersos y formar un único bloque de memoria de gran tamaño.

Sin embargo, esta operación resulta costosa para el sistema, ya que reorganizar toda la información interrumpe las tareas y consume valioso tiempo de procesamiento.

Problemas en la memoria virtual

La implementación de la memoria virtual trae su propio grupo de problemas al permitir que los procesos usen un espacio mayor a la RAM física disponible.

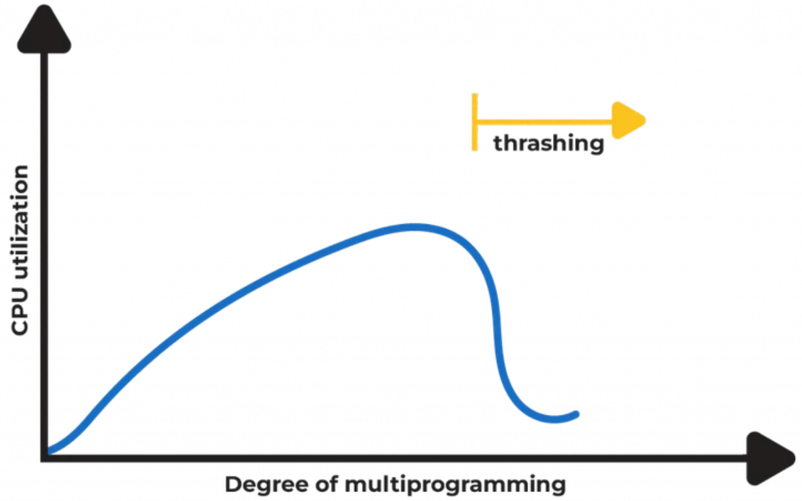
El sistema debe alternar páginas de información entre la memoria principal y el almacenamiento secundario, provocando lo que se conoce como "fallo de página" cada vez que un proceso necesita un dato que no está cargado, obligando al sistema a ir a buscarlo.



Riesgo de thrashing

Si los fallos de página ocurren con demasiada frecuencia, el equipo sufre un colapso crítico conocido como thrashing o trasiego.

En este estado, el sistema pasa casi todo su tiempo intercambiando páginas entre el disco y la memoria principal, perdiendo por completo la capacidad de ejecutar los procesos reales y provocando que el rendimiento general de la computadora disminuya drásticamente.



Referencias.

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). Fundamentos de sistemas operativos (7.ª ed.). McGraw-Hill Interamericana.

2. Stallings, W. (2005). Sistemas operativos: Aspectos internos y principios de diseño (5.ª ed.). Pearson Educación.

3. Tanenbaum, A. S. (2009). Sistemas operativos modernos (3.ª ed.). Pearson Educación de México.

4. Carretero Pérez, J., García Carballeira, F., de Miguel Anasagasti, P., y Pérez Costoya, F. (2001). Sistemas operativos: Una visión aplicada. McGraw-Hill/Interamericana de España.

5. La Red Martínez, D. L. (2004). Sistemas operativos. Universidad Nacional del Nordeste, Facultad de Ciencias Exactas y Naturales y Agrimensura, Departamento de Informática.

6. Perdomo Bonilla, J. V. (2017). Sistemas operativos móviles [Presentación]. Universidad Surcolombiana.

Glosario